



Creating Plugins with Claude v2.5

Creating Freque Plugins with Claude v2.5

Complete guide for using Claude to create audio-reactive visualizations

What's New in v2.5

- **Window Resize Requirements** - How to request proper resize handling for plugins ★ NEW
- **Correct Dropdown Format** - Proper syntax with value/label objects ★ NEW
- **Audio Integration Clarity** - How to properly access bass/mid/treble/beat ★ NEW
- **Variable Initialization Pattern** - Critical constructor setup ★ NEW
- **Control Effectiveness Tips** - Ensuring controls actually work ★ NEW
- **Audio Reactivity Patterns** - Base + multiplier for all audio levels ★ NEW

What's New in v2.4

- **Advanced Retro Effects** - Pixelation, color banding, enhanced RGB shift with examples ★ NEW

- **Ring Spawning Pattern** - How to request continuous tunnel effects ★ NEW
- **Granular Audio Controls** - Requesting sensitivity, smoothing, and per-feature intensity ★ NEW
- **Frequency-Specific Reactivity** - Mapping bass/mid/treble to specific effects ★ NEW
- **Custom Dial Fill Colors** - Requesting per-plugin dial color customization ★ NEW

What's New in v2.3

- **Extended Color Palettes** - 10 retro schemes (C64, Game Boy, CRT, etc.) ★ NEW
- **Film Grain Effects** - Requesting clustered noise and VHS artifacts ★ NEW

What's New in v2.2

- **WebGL/WebGL2 Plugin Prompting** - How to request shader-based visualizations ★ NEW
- **Procedural Effect Patterns** - Prompting for GPU-accelerated effects ★ NEW
- **PSYCH Plugin as Reference** - Real-world WebGL2 example ★ NEW

What's in v2.1

- **Advanced Control Patterns** - Non-linear scaling and source-dependent adjustments

What's in v2.0

- **Video Reflection Prompting** - How to request cube camera systems
- **Advanced Three.js Requests** - Complex 3D scene prompting
- **Performance Considerations** - Asking for optimized implementations

- **Dual Mode Patterns** - Requesting multiple rendering modes
-

Table of Contents

1. [Quick Start](#)
 2. [The Universal Plugin Prompt](#)
 3. [Requesting 2D Canvas Plugins](#) ★ NEW
 4. [Requesting WebGL/WebGL2 Plugins](#) ★ NEW
 5. [Requesting Three.js Plugins](#)
 6. [Video Reflection Systems](#)
 7. [Common Request Patterns](#)
 8. [Troubleshooting](#)
-

Quick Start

Your First Plugin in 10 Minutes

Step 1: Copy the Template

Claude, create a Freque plugin:

Plugin Name: particleburst

Type: 2D Canvas

Visual Description:

Colorful particles that burst outward from the center on beat drops.

Audio Reactivity:

- Bass: Triggers particle bursts
- Mid: Controls particle speed
- Treble: Changes particle colors

Controls:

- Particle Count: dial, 50-500, default 200
- Burst Size: dial, 1.0-5.0, default 2.0
- Color Scheme: dropdown with dropdown-selector-mixer class, [Rainbow, Warm, Cool, Monochromatic]
- Auto Burst: checkbox toggle with btn-primary-mixer class, starts OFF, automatic particle bursts

Presets:

1. Gentle: Small bursts, slow speed, cool colors
2. Intense: Large bursts, fast speed, warm colors

Step 2: Send to Claude

Copy the filled template and send it. Claude will generate complete, working code.

Step 3: Save the File

Save as `particleburst-freque-plugin.js` in the `js/plugins/` directory.

Step 4: Reload Freque

Refresh your browser. The plugin appears in the mixer!

Critical Requirements (v2.5)

Dropdown Controls **MUST** Use Correct Format

WRONG - Will Not Work:

```
options: ['Option 1', 'Option 2'] // Simple array
options: { value1: 'Label 1' }    // Object format
```

CORRECT - Only This Works:

```
this.addControl('colorScheme', {
  type: 'dropdown',
  label: 'Color Scheme',
  className: 'dropdown-selector-mixer', // REQUIRED
  options: [
    { value: 'classic', label: 'Classic' },
    { value: 'neon', label: 'Neon' }
  ],
  value: 'classic',
  onChange: (value) => {
    this.colorScheme = value;
  }
});
```

Initialize ALL Variables in Constructor

✗ WRONG - Variables Undefined:

```
constructor() {
  this.setupControls(); // size is undefined!
}
```

✓ CORRECT - Variables Initialized:

```
constructor() {
  this.size = 50;           // Initialize first
  this.speed = 1.0;
  this.colorScheme = 'classic';
  this.setupControls();
}
```

Audio Data Access

✓ Use This Pattern:

```
onUpdate(deltaTime, timestamp, sharedAudioData) {  
    this.audioLevel = this.getAudioEnergy();  
    this.bassLevel = sharedAudioData.bass || 0;  
    this.midLevel = sharedAudioData.mid || 0;  
    this.trebleLevel = sharedAudioData.treble || 0;  
    this.beatDetected = sharedAudioData.beat || false;  
}
```

Controls Must Actually Work

When you add a control, the variable must be USED:

```
// Variable is set...  
onChange: (value) => { this.speed = value; }  
  
// ...and USED in rendering  
onUpdate() {  
    this.position += this.speed * deltaTime; // ✓ Used here  
}
```

Window Resize Handling

For Game Plugins or Size-Dependent Visualizations:

Plugins that have positioned elements (aliens, obstacles, particles tied to canvas size) **must implement** `onResize()` :

```
onResize(width, height) {  
    // 1. Update center/positioning variables  
    this.centerX = width / 2;  
    this.player.x = width / 2;  
  
    // 2. Recreate size-dependent elements  
    this.recreateEnemies();  
    this.recreateObstacles();  
  
    // 3. PRESERVE GAME STATE (score, lives, etc.)  
}  

```

When Requesting Plugins from Claude:

If your plugin has game elements or positioned objects, **include this in your request:**

RESIZE HANDLING:

- Implement onResize() to handle window resize
 - Recreate enemies/obstacles/stars for new canvas size
 - Preserve game state (score, lives, wave) during resize
 - Reposition player/ship to center of new canvas
-

The Universal Plugin Prompt

Complete Template

Claude, help me create a Freque plugin:

BASIC INFO

Plugin Name: [one-word-or-hyphenated]

Type: [2D Canvas / WebGL Shader / WebGL2 Shader / 3D Three.js]

Description: [one sentence description]

VISUAL DESCRIPTION

Main Elements:

[Describe what you see - particles, shapes, lines, etc.]

Movement Style:

[How things move - float, spin, pulse, wave, etc.]

Color Scheme:

[Colors and how they change with audio]

Layout:

[Where things appear - center, edges, random, grid, etc.]

Special Effects:

[Any special visual effects - glow, trails, fade, blur, etc.]

AUDIO REACTIVITY

Bass Response (Low Frequencies):

[What bass affects - size, quantity, movement speed, etc.]

Mid Response (Mid Frequencies):

[What mids affect - rotation, color, spread, etc.]

Treble Response (High Frequencies):

[What treble affects - brightness, detail, shimmer, etc.]

Beat Detection:

[What happens on beat drops - burst, pulse, flash, etc.]

CONTROLS

IMPORTANT:

- Toggle controls use checkbox type with className: 'btn-primary-mixer'
- Numeric controls use dial type (not slider)
- Dropdowns must include className: 'dropdown-selector-mixer' AND use correct options for

Dials (numeric controls):

- [Name]: dial, [min]-[max], default [value], [what it controls]

Dropdowns (CRITICAL - must use correct format):

- [Name]: dropdown with dropdown-selector-mixer class, options as [{ value: 'x', label: 'x' }]
- Example: Color Scheme: dropdown with dropdown-selector-mixer class, options as [{ value: 'x', label: 'x' }]

Toggles (ON/OFF switches - must include className):

- [Name]: checkbox toggle with btn-primary-mixer class, starts [ON/OFF], [what it toggles]
- Example: Audio Reactive: checkbox toggle with btn-primary-mixer class, starts ON, enables
- Example: Color Morph: checkbox toggle with btn-primary-mixer class, starts OFF, smooths

Mode Buttons (switches between modes):

- [Name]: button toggle, switches between [Mode A / Mode B]

Action Buttons (one-time actions):

- [Name]: button, [what it does when clicked]

PRESETS (minimum 2)

Preset 1 - [Name]:

Purpose: [What music/mood this suits]

Settings: [Key control values]

Preset 2 - [Name]:

Purpose: [What music/mood this suits]

Settings: [Key control values]

Preset 3 - [Name] (optional):

Purpose: [What music/mood this suits]

Settings: [Key control values]

SPECIAL REQUESTS

[Any specific techniques, technologies, references]

[Performance requirements]

[Compatibility needs]

[Custom dial fill color (CSS variable name, e.g., '--accent-color')]

RESIZE HANDLING (For Game/Positioned Elements):

[If your plugin has positioned game elements (aliens, obstacles, etc.)]

- Implement onResize() to recreate elements for new canvas size
- Preserve game state (score, lives, wave) during resize
- Reposition player/ship to center of new canvas
- Update boundaries for collision detection

Requesting Three.js Plugins

Basic Three.js Request

Claude, create a Three.js plugin for Freque:

Plugin Name: floatingspheres

Type: 3D Three.js

Create floating metallic spheres that reflect the video/background.

Use proper Freque Three.js setup with:

- Container-based sizing (not canvas buffer)
- Individual CSS property assignment
- Proper environment mapping from video or background image

Important: Three.js Setup Requirements

ALWAYS include this in Three.js requests:

CRITICAL Three.js Requirements:

1. Get dimensions from #visualizationContainer element
2. Use manual CSS property assignment (NOT cssText copying)
3. Set canvas CSS to width: 100% and height: 100%
4. Use devicePixelRatio for high-DPI displays
5. Set output encoding to sRGBEncoding

Claude has been trained on these patterns and will implement them correctly.

Requesting 2D Canvas Plugins

When to Use 2D Canvas

Best for: - Simple visualizations (waveforms, spectrum analyzers) - Drawing shapes, text, images - Particle systems in 2D space - Classic canvas drawing API effects

2D Canvas Plugin Template

Claude, create a 2D canvas Freque plugin:

Plugin Name: waveform-pulse

Type: 2D Canvas

Visual Description:

An animated waveform that draws across the screen from left to right. The waveform height responds to audio amplitude. Background fades slowly creating a trailing effect.

Audio Reactivity:

- Bass: Controls waveform thickness
- Mid: Controls waveform amplitude
- Treble: Changes waveform color (hue shift)
- Beat detection: Creates a pulse flash

Controls:

- Line Thickness: dial, 1-10, default 3
- Smoothness: dial, 0.1-1.0, default 0.5
- Color Scheme: dropdown with dropdown-selector-mixer class, [Rainbow, Warm, Cool, Monoch
- Fade Speed: dial, 0.01-0.5, default 0.1

Presets:

1. Smooth: Thin lines, high smoothness, cool colors, slow fade
2. Aggressive: Thick lines, low smoothness, warm colors, fast fade

2D Canvas Key Points

What Claude will use: - `this.ctx` for 2D canvas context (auto-created) - Canvas drawing methods: `fillRect` , `strokeRect` , `beginPath` , `lineTo` , etc. - `this.canvas.width` and `this.canvas.height` for dimensions - Standard canvas compositing and blend modes

Example request phrases: - “Draw circles using canvas arc” - “Use fillRect to create bars” - “Apply motion blur with low alpha clearRect” - “Use canvas gradients for color effects”

Requesting WebGL/WebGL2 Plugins

When to Use WebGL/WebGL2

Best for: - **Procedural shader effects** (liquid simulations, fractals, noise) - **GPU-accelerated effects** (thousands of particles, complex calculations) - **Fullscreen shader art** (raymarching, domain warping, procedural textures) - **Advanced post-processing** (blur, bloom, distortion)

Browser Support: - WebGL 1.0: ~97% (excellent compatibility) - WebGL 2.0: ~95% (good compatibility)

WebGL2 Plugin Template (Advanced Shaders)

Claude, create a WebGL2 shader-based Freque plugin:

Plugin Name: liquid-flow

Type: WebGL2 Shader

Visual Description:

Procedural liquid/bubble effect using domain warping and metaballs.

Smooth organic flowing shapes with bright edge highlights and iridescent shimmer effects. Think lava lamp meets soap bubbles.

Shader Technique:

- Fullscreen fragment shader (GLSL 3.0 ES)
- Perlin noise for organic flow
- Domain warping for complex motion
- Metaball/SDF-based bubble shapes
- Smooth color gradients

Audio Reactivity:

- Bass: Modulates bubble density
- Mid: Controls flow speed
- Treble: Affects edge brightness
- Beat detection: Triggers color shifts

Controls:

- Animation Speed: dial, 0-3, default 1.0
- Bubble Density: dial, 0.5-2.5, default 1.5
- Flow Complexity: dial, 0.2-2.0, default 1.0
- Edge Brightness: dial, 0-5, default 1.5
- Color Scheme: dropdown with dropdown-selector-mixer class, [Acid Orange/Cyan, Blue Bubble]
- Zoom: dial, 1-10, default 4.0

Technical Requirements:

- WebGL2 context with high precision
- Fullscreen quad rendering
- Smooth noise functions

- Multiple time variables for independent animation
- Proper DPI scaling for 4K support

Performance Target:

- 60fps at 4K resolution
- Optimized fragment shader
- Minimal uniform updates

WebGL Plugin Template (Better Compatibility)

Claude, create a WebGL shader-based Freque plugin:

Plugin Name: particle-galaxy

Type: WebGL Shader

Visual Description:

GPU-accelerated particle system creating a galaxy spiral effect. Thousands of particles orbiting around a center point with depth-based opacity and color variations.

Shader Technique:

- GLSL 1.0 for compatibility
- Point sprites for particles
- Vertex shader for position calculations
- Fragment shader for color/alpha

Audio Reactivity:

- Bass: Expands/contracts the galaxy
- Mid: Controls rotation speed
- Treble: Affects particle brightness

Controls:

- Particle Count: slider, 1000-50000, default 10000
- Rotation Speed: slider, -5 to 5, default 1.0
- Spread: slider, 1-10, default 5.0
- Particle Size: slider, 1-10, default 3.0

Technical Requirements:

- WebGL 1.0 context for maximum compatibility
- Vertex buffer for particle positions
- Uniform updates for audio reactivity
- Point sprite rendering

What Claude Will Implement for WebGL Plugins

WebGL2 Shader Plugin Setup

1. Context Creation

```
this.gl = this.canvas.getContext('webgl2', {  
  alpha: false,  
  antialias: true,  
  powerPreference: 'high-performance'  
});
```

2. Shader Compilation (GLSL 3.0 ES)

```
#version 300 es  
precision highp float;  
  
uniform vec2 resolution;  
uniform float time;  
out vec4 fragColor;  
  
void main() {  
  vec2 uv = gl_FragCoord.xy / resolution;  
  fragColor = vec4(uv, 0.5, 1.0);  
}
```

3. Fullscreen Quad

- Vertex buffer with quad covering -1 to 1
- Triangle strip rendering

4. Uniform Management

- Resolution, time, audio data

- User control values
- Color scheme parameters

5. Resize Handling

- DPI-aware canvas sizing
- Viewport updates
- 4K-ready resolution

Key Differences: WebGL vs WebGL2

Feature	WebGL 1.0	WebGL2
GLSL Version	1.0	3.0 ES
Syntax	<code>attribute , varying</code>	<code>in , out</code>
Output	<code>gl_FragColor</code>	<code>out vec4 fragColor</code>
Loops	Limited	Full support
3D Textures	✗ No	✓ Yes
Integer Textures	✗ No	✓ Yes
Transform Feedback	✗ No	✓ Yes

WebGL Plugin Request Patterns

For Procedural Effects

Claude, create a WebGL2 plugin with:

- Raymarching distance field for 3D-looking effects
- Perlin noise for organic motion
- Domain warping for complex patterns
- Smooth color palette using cosine gradients
- Audio-reactive parameters (time scale, density, intensity)

For Particle Systems

Claude, create a WebGL plugin with:

- Vertex shader for particle position calculations
- Point sprites for rendering
- 10,000+ particles for dense effects
- Audio-reactive forces (attraction, repulsion, swirl)
- Color variation based on velocity/position

For Post-Processing

Claude, create a WebGL2 plugin with:

- Render-to-texture for multi-pass effects
- Gaussian blur implementation
- Bloom/glow effect
- Chromatic aberration
- Audio-reactive distortion

Critical WebGL Plugin Requirements

ALWAYS specify in your request:

1. **Context type:** “WebGL2” or “WebGL 1.0”
2. **Shader technique:** What shader effect you want
3. **Audio reactivity:** Which audio bands affect what
4. **Performance target:** “60fps at 4K” or similar

Example Critical Request:

IMPORTANT TECHNICAL REQUIREMENTS:

- Use WebGL2 context (`this.gl = this.canvas.getContext('webgl2')`)
- Create context in `onInitialize()` BEFORE accessing `this.ctx`
- Override `shouldClearCanvas()` to return false
- Use GLSL 3.0 ES syntax (`#version 300 es`)
- Handle DPI scaling for high-resolution displays
- Target 60fps at 4K resolution

Reference Examples

Simple WebGL2 Template Request

Claude, create a basic WebGL2 test plugin:

- Fullscreen gradient that changes color with audio
- Fragment shader only (simple quad rendering)
- Color changes based on bass, mid, treble
- One slider for animation speed
- Minimal code, well-commented

Advanced WebGL2 Request (PSYCH-style)

Claude, create an advanced WebGL2 liquid bubble visualizer:

Shader Technique:

- Metaball-based bubble field generation
- Multi-octave Perlin noise for organic flow
- Domain warping for complex motion patterns
- Smooth color gradients with multiple schemes
- Edge detection for bright bubble outlines
- Interior shimmer effects using noise

Technical Requirements:

- WebGL2 with GLSL 3.0 ES shaders
- Multiple independent time variables
- Perlin noise implementation in shader
- Smooth random functions for animation morphing
- Support for 5+ color schemes
- Audio-reactive density, edge brightness

Controls:

- Animation Speed: 0-3
- Rotation Speed: -20 to 20
- Bubble Density: 0.3-2.5
- Flow Complexity: 0.2-2.0
- Bubble Separation: 1.5-8.0
- Background Speed/Intensity
- Edge Brightness: 0-5
- Color Intensity: 0.5-3
- Color Scheme dropdown
- Color/Animation Morph toggles with speed controls

Performance:

- Optimize for 4K/60fps
- Efficient noise calculations
- Minimal uniform updates

Custom Dial Fill Colors

Requesting Custom Dial Colors

If you want all dials in your plugin to use a specific color from the theme:

```
Claude, create a plugin with custom dial fill colors:
```

```
Dial Color:
```

- Use `--accent-color` for all dial fills
- Dial outlines should remain default (unchanged)
- Color should change automatically with theme

Available CSS Variables: - `--accent-color` - Theme accent color (recommended)
- `--highlight-color` - Theme highlight color - `--error-color` - Error/warning color
- `--success-color` - Success color - `--warning-color` - Warning color - Any other CSS variable defined in your theme

Example Request:

```
Claude, create a plugin where all dials use the accent color:
```

```
Plugin Name: colorfulviz
```

```
Type: 2D Canvas
```

```
Special Requests:
```

- All dials should use `--accent-color` for fill color
- This makes the plugin's dials match the theme accent color

What Claude Will Implement:

```
super('colorfulviz', visualizer, {  
  version: '1.0.0',  
  dialFillColor: '--accent-color' // All dials will use accent color  
});
```

Benefits: - Visual consistency with theme - Easy to identify plugin dials - Automatic theme support - No CSS changes needed

Requesting Three.js Plugins

When to Use Three.js

Best for: - 3D scenes with meshes, lights, cameras - Video reflections (cube camera systems) - Complex 3D geometry and materials - Environment mapping and PBR rendering

Video Reflection Systems

When to Request Cube Cameras

If you want **video reflections on chrome/metallic surfaces**, you MUST request a cube camera system:

Claude, create a Three.js plugin with chrome spheres reflecting the video.

IMPORTANT:

- Use cube camera system for video reflections (required for Three.js r120+)
- Create inverted video sphere for cube camera to capture
- Update cube camera every frame to keep reflections synchronized
- Use layer management to hide/show video sphere

Video Reflection Request Template

Claude, create a reflective metallic [object] plugin:

Technology Requirements:

- Three.js with cube camera system for video reflections
- Support both video (#bgVideo) and background image (#bgImage) sources
- Dual mode: direct texture AND chrome reflection
- Dynamic cube camera positioning for realistic reflections

Visual:

- [Describe your reflective objects]
- Chrome/metallic material
- Reflections show the video/background

Controls:

- Toggle between direct texture and reflection modes
- Toggle between video and background image sources
- Metalness: 0.0-1.0, default 1.0
- Roughness: 0.0-1.0, default 0.0
- Reflection Intensity: 0.0-3.0, default 0.5

Performance:

- Use shared cube camera (not per-object)
- 2048 cube map resolution for balance
- Only update cube camera when in reflection mode

What Claude Will Implement

When you request a video reflection system, Claude will create:

1. **Cube Render Target**

- High resolution (2048 or 4096)

- RGBA format with mipmaps
- sRGB encoding

2. **Cube Camera**

- Positioned strategically (often at origin or average object position)
- Enabled to see hidden layer
- Updates every frame

3. **Large Inverted Video Sphere**

- 500 unit radius
- Video texture with EquirectangularReflectionMapping
- Hidden from main camera via layer system
- Proper rotation for correct reflections

4. **Material Setup**

- envMap using cube map texture
- High metalness (0.8-1.0)
- Low roughness (0.0-0.2)
- Adjustable envMapIntensity

5. **Update Loop**

- Video texture needsUpdate every frame
- Cube camera update when in reflection mode
- Temporary object hiding during capture

Cube Camera Prompting Examples

Example 1: Chrome Spheres

Claude, create chrome spheres that reflect the video input.

Requirements:

- Use cube camera for video reflections
- Support both video and background image
- Dynamic cube camera positioning (average sphere position)
- Layer system to hide/show video sphere
- Proper sRGB encoding throughout

Spheres:

- 10-30 spheres
- Floating animation
- Audio-reactive size (bass)
- Ultra chrome appearance

Example 2: Reflective Floor

Claude, create a reflective floor plugin showing video reflections.

Requirements:

- Large plane geometry for floor
- Cube camera positioned above floor
- Video sphere for reflections
- Support toggling between mirror and matte

Floor:

- Positioned below camera view
- PBR material (metalness + roughness)
- Grid lines for reference
- Audio-reactive ripple effect

Example 3: Mirror Ball

Claude, create a disco mirror ball with video reflections.

Requirements:

- Single large sphere at center
- Cube camera at sphere center
- Ultra-high quality reflections (4096 cube map)
- Rotation animation
- Beat-reactive spin speed

Performance Considerations

Requesting Optimized Cube Cameras

Claude, create [plugin] with video reflections.

Performance Requirements:

- Use shared cube camera (not per-object)
- 1024 or 2048 cube map resolution
- Update cube camera every other frame
- Only update when video is playing
- Dispose resources properly in cleanup

Quality vs Performance Trade-off

You can specify:

Cube Map Quality:

- 4096 resolution for ultra quality (expensive)
- 2048 resolution for high quality (balanced) ← RECOMMENDED
- 1024 resolution for good quality (performance)

Update Frequency:

- Every frame for perfect sync (expensive)
- Every 2-3 frames for good balance ← RECOMMENDED
- Every 5 frames for performance

Common Request Patterns

Requesting Advanced Retro Effects

Pixelation & Color Banding

When you want authentic low-resolution retro aesthetics:

Claude, create a pixelation effect for retro 8-bit/16-bit look:

Pixelation Control (dial 0-100):

- 0 = Full resolution (modern, crisp)
- 25 = 640x480 resolution (VGA era)
- 50 = 320x240 resolution (console era)
- 100 = 160x120 resolution (classic arcade)

Implementation:

- Downscale canvas to target resolution with `imageSmoothingEnabled = false`
- Upscale back with nearest-neighbor filtering
- Apply LAST in render pipeline (after all other effects)

Color Banding Control (dial 0-100):

- 0 = 24-bit color (16.7 million colors)
- 50 = 4-bit color (16 colors per channel)
- 100 = 1-bit color (2 colors per channel, extreme posterization)

Implementation:

- Use `getImageData/putImageData` to quantize RGB values
- Apply BEFORE vignette/curvature (to avoid posterizing gradients)

These effects create that authentic retro game console/arcade look!

Ring Spawning System

For continuous tunnel effects like Tempest:

Claude, create a tunnel effect using a ring spawning system:

Ring Lifecycle:

- Rings spawn at center every 0.3 seconds
- Each ring lives for 3 seconds (age from 0 to 1)
- Rings scale from small (age=0) to large (age=1)
- Rings fade out as they age ($\alpha = 1 - \text{age}$)
- Dead rings (age ≥ 1) are removed from array

Implementation:

- Store rings in array: [{ age: 0-1, rotation: angle }, ...]
- In onUpdate: spawn new rings, age existing rings, remove dead ones
- In onRender: draw all active rings from oldest to newest

Audio Reactive:

- Bass → Faster spawn rate (shorter interval)
- Bass → Ring size pulsing
- Beat detection → Spawn burst of rings

This creates smooth, organic tunnel effects with proper depth!

Color Morphing

For smooth, dynamic color transitions:

Claude, add color morphing to cycle through colors smoothly:

Color Morph (checkbox toggle):

- Toggle to enable/disable color morphing
- Starts OFF by default

Morph Speed (dial 0-5.0):

- Controls how fast colors transition
- 1.0 = normal speed
- 5.0 = very fast

Implementation:

- Use HSL color space for smooth hue transitions
- Increment hue over time: `hue += speed * deltaTime * 0.05`
- Wrap hue at 360 degrees
- Can combine with palette cycling (interpolate between preset colors)
- Can make audio-reactive (bass affects morph speed)

Example pattern:

```
if (this.colorMorph) {  
    this.hue += this.morphSpeed * deltaTime * 0.05;  
    if (this.hue > 360) this.hue -= 360;  
}  
const color = `hsl(${this.hue}, 100%, 50%)`;
```

This creates flowing, organic color changes that enhance visual appeal!

Enhanced VHS/CRT Effects

Request improved glitch effects:

Claude, create enhanced VHS/RGB shift effects:

RGB Chromatic Aberration:

- Horizontal shift (left/right for R/B channels)
- Vertical shift (up/down component, half of horizontal)
- Edge fringing (magenta/cyan at high intensity + audio)
- Audio-reactive (treble increases shift amount)

Film Grain Static:

- Clustered noise (Perlin-like) for film grain feel
- 70% grayscale static (traditional)
- 30% colored noise (VHS artifact)
- Variable pixel size (1-2px for organic look)
- Audio-reactive density (treble increases particle count)

VHS Glitch:

- Random horizontal displacement lines
- Tracking errors (horizontal black lines)
- Triggered by audio treble peaks
- Temporary glitches that fade after 1 second

These create that authentic damaged VHS tape aesthetic!

Advanced Audio Reactivity

Granular Audio Controls

Request fine-tuned control over how audio affects visuals:

Claude, implement advanced audio reactivity with granular controls:

Global Controls:

- Audio Sensitivity (dial 0-200%): Global multiplier for all audio
- Audio Smoothing (dial 0-100%): Exponential smoothing factor

Per-Feature Intensity Controls:

- Color Intensity (dial 0-200%): How much audio affects color
- Speed Intensity (dial 0-200%): How much audio affects speed
- Segment Intensity (dial 0-200%): How much audio affects segment count

Frequency-Specific Toggles:

- Bass → Rotation (checkbox): Bass controls rotation speed
- Bass → Spawn (checkbox): Bass controls ring spawn rate
- Mid → Glow (checkbox): Mids control glow intensity
- Treble → Thickness (checkbox): Treble controls line thickness

Implementation:

- Apply sensitivity multiplier in `getAudioData()`
- Use smoothing factor for exponential smoothing
- Multiply per-feature intensity when applying effects
- Only apply frequency-specific effects when toggled on

Default State:

- All controls at 100% (neutral)
- Only essential toggles ON by default
- Advanced toggles OFF by default

This gives users complete control over audio reactivity!

Frequency-Specific Mapping

Request specific frequency ranges mapped to specific effects:

Claude, map specific frequencies to specific visual effects:

Bass (Low Frequencies):

- Physical movement (rotation, position, spawning)
- "Heavy" effects that feel impactful
- Example: Bass → Rotation speed, Bass → Ring spawn rate

Mids (Middle Frequencies):

- Visual intensity (glow, brightness, saturation)
- "Energy" effects that pulse
- Example: Mid → Glow intensity, Mid → Color saturation

Treble (High Frequencies):

- Fine details (thickness, particle count, shimmer)
- "Sparkle" effects that add excitement
- Example: Treble → Line thickness, Treble → Static amount

Implementation:

- Each effect has a toggle control (OFF by default)
- When ON, specific frequency affects specific visual
- Smooth transitions using exponential smoothing
- Can combine multiple frequency mappings

Best Practice:

- Bass for movement/physics
- Mids for intensity/energy
- Treble for details/shimmer

This creates natural, intuitive audio-visual mapping!

Advanced Control Patterns

Non-Linear Control Scaling

When you need finer control in certain ranges, request two-part scaling:

Claude, create a Float Speed control with non-linear scaling:

Control: Float Speed slider (0-10)

Scaling Behavior:

- 0-3: Maps to 0-1.0x speed (slow motion to normal)
- 3: Center point = 1.0x normal speed (easy reset)
- 3-10: Maps to 1.0-2.0x speed (normal to double speed)

Implementation:

```
const speedFactor = value <= 3
  ? value / 3.0
  : 1.0 + ((value - 3) / 7.0);
```

Why: Provides finer precision in the slow-motion range where users spend more time, while still allowing high-speed effects.

When to Use This Pattern: - Any control where one range needs more precision - Controls with a “normal” center point - Speed/rate controls where 0 should be “frozen” - Intensity controls where subtle changes matter more at low values

Example Request:

Claude, create an intensity control:

- Range: 0-10
- 0-5: Fine control (maps to 0-0.5)
- 5: Normal intensity (0.5)
- 5-10: Coarse control (maps to 0.5-1.0)

Use two-part scaling for better UX.

Source-Dependent Material Adjustments

Request automatic adjustments when different sources need different handling:

Claude, create reflective spheres that support both video and images:

Auto-adjust brightness:

- Video reflections: Use 80% of set intensity (they render brighter)
- Image reflections: Use 100% of set intensity
- User sets one value, system adjusts automatically
- Transparent to user (no extra controls)

Implementation:

```
if (sourceType === 'video') {  
    material.envMapIntensity = userValue * 0.8;  
} else {  
    material.envMapIntensity = userValue;  
}
```

Why Request This: Different texture sources (video vs static images) can render with different characteristics. Automatic adjustments maintain visual consistency without requiring users to manage multiple controls.

2D Canvas Plugins

Particle Systems

Claude, create a 2D particle system:

Particles:

- 500-2000 particles
- Emit from center/edges/random
- Float/drift/burst patterns
- Size based on bass
- Color based on treble
- Fade trails for motion blur

Audio:

- Bass triggers emission
- Mids control speed
- Treble affects color

Controls:

- Particle count
- Emission rate
- Particle lifetime
- Trail length
- Color scheme

Waveforms

Claude, create an audio waveform visualizer:

Waveform:

- Circular/linear layout
- Smooth curves
- Line or bar style
- Gradient colors
- Glow effect

Audio:

- Use actual waveform data
- Bass emphasizes low frequencies
- Treble emphasizes high frequencies

Controls:

- Waveform style (line/bars)
- Color scheme
- Thickness
- Smoothing

3D Three.js Plugins

Geometric Shapes

Claude, create 3D geometric shapes:

Shapes:

- Cubes, spheres, toruses, etc.
- Multiple objects in scene
- Rotation animations
- Audio-reactive scale
- PBR materials

Controls:

- Shape type selector
- Material properties
- Animation speed
- Quantity
- Arrangement (grid/random/orbit)

Tunnel/Corridor Effects

Claude, create a tunnel effect:

Tunnel:

- Repeating rings/segments
- Forward motion illusion
- Texture or geometric
- Audio pulses through tunnel
- Camera movement

Audio:

- Bass expands rings
- Beat creates pulse wave
- Treble affects texture detail

Dual Mode Patterns

Claude, create a plugin with multiple render modes:

Modes:

1. Particles (2D canvas)
2. Geometry (3D Three.js)
3. Waveform (2D canvas)

Each mode:

- Shares same controls where applicable
- Different visual style
- Toggle between modes with button

Controls:

- Mode selector
- [Shared controls]
- [Mode-specific controls]

Troubleshooting

Common Issues and How to Ask Claude

Issue: “WebGL2 not supported” error

Ask Claude:

Claude, I'm getting "WebGL2 not supported" error but my browser supports it.

Please verify:

1. Context is created in `onInitialize()` before accessing `this.ctx`
2. Using `this.gl = this.canvas.getContext('webgl2', {...})`
3. Context creation happens BEFORE any `this.ctx` access
4. `shouldClearCanvas()` returns false
5. No 2D context is created accidentally

Also please add a proper fallback message to the user if WebGL2 truly isn't supported.

Issue: Black screen / nothing rendering (WebGL)

Ask Claude:

Claude, my WebGL plugin shows a black screen.

Please check:

1. Shader compilation successful (check for errors in compile log)
2. Shader linking successful (check link status)
3. Uniforms are being set in `onRender`
4. Vertex buffer is bound correctly
5. Drawing command is called (`gl.drawArrays`)
6. Viewport is set to correct dimensions
7. Clear color is set if needed

Add `console.log` statements for debugging each step.

Issue: Shader compile errors

Ask Claude:

Claude, I'm getting shader compilation errors.

For WebGL2:

- Ensure using `#version 300 es`
- Use `'in'` and `'out'` instead of `'attribute'` and `'varying'`
- Use `'texture'` instead of `'texture2D'`
- Declare output: `out vec4 fragColor`

For WebGL 1.0:

- Don't use `#version` directive
- Use `'attribute'` and `'varying'`
- Use `'texture2D'` for sampling
- Use `gl_FragColor` for output

Please check and fix the shader syntax for [WebGL/WebGL2].

Issue: Low resolution / blurry (WebGL)

Ask Claude:

Claude, my WebGL plugin looks blurry or low resolution.

Please add proper DPI scaling:

1. Multiply canvas width/height by `devicePixelRatio`
2. Set `gl.viewport` to actual canvas dimensions
3. Pass actual pixel dimensions to resolution uniform
4. Update resize handling to maintain DPI scaling

Target: Should look sharp on 4K displays and Retina screens.

Issue: Video reflections not working

Ask Claude:

Claude, the video reflections aren't showing on my chrome spheres.
Can you:

1. Verify cube camera setup is correct
2. Check video sphere is on layer 1
3. Confirm cube camera update is in onRender
4. Check video element is playing
5. Verify sRGB encoding is consistent

Issue: Canvas sizing wrong

Ask Claude:

Claude, the Three.js canvas isn't sizing correctly.
Please verify:

1. Using visualizationContainer for dimensions
2. Setting individual CSS properties (not cssText)
3. Using percentages for width/height
4. Setting devicePixelRatio properly

Issue: Textures upside down

Ask Claude:

Claude, the video/image textures appear upside down.
Please:

1. Set flipY = true on textures
2. Remove any rotation-based fixes
3. Check mapping type is correct

Issue: Dropdowns not displaying properly

Ask Claude:

Claude, my dropdowns aren't styled correctly or aren't visible.

Please ensure:

1. `className: 'dropdown-selector-mixer'` is specified in `addControl`
2. The dropdown has proper options array with value/label pairs
3. `onChange` handler is implemented

Example:

```
this.addControl('colorScheme', {
  type: 'dropdown',
  label: 'Color Scheme',
  className: 'dropdown-selector-mixer', // REQUIRED
  options: [
    { value: 0, label: 'Option 1' },
    { value: 1, label: 'Option 2' }
  ],
  value: 0,
  onChange: (value) => {
    this.colorScheme = parseInt(value);
  }
});
```

Issue: Toggle controls not working

Ask Claude:

Claude, my toggle controls (ON/OFF switches) aren't working.

Please use this EXACT pattern for toggles:

Use type: 'checkbox' (NOT type: 'button'):

```
this.myToggle = false;
this.addControl('myToggle', {
  type: 'checkbox',          //  Use checkbox for toggles
  label: 'My Toggle',
  checked: false,         // Initial state
  onChange: (value) => {  //  Use onChange
    this.myToggle = value;
    console.log('Toggle:', value);
  }
});
```

For toggles that start ON:

```
this.audioReactive = true;
this.addControl('audioReactive', {
  type: 'checkbox',
  label: 'Audio Reactive',
  checked: true,          // Starts ON
  onChange: (value) => {
    this.audioReactive = value;
  }
});
```

IMPORTANT:

- Use type: 'checkbox' for ON/OFF toggles
- Use type: 'button' only for mode switching or one-time actions
- No manual DOM manipulation needed - checkbox handles state automatically

Issue: Button labels not updating

Ask Claude:

```
Claude, button labels aren't updating when I click them.  
Please use direct DOM queries with data-control attributes:  
  
const button = document.querySelector('[data-control="controlName"]');  
button.textContent = newLabel;
```

Issue: Performance problems

Ask Claude:

```
Claude, the plugin is running slowly.  
Please optimize:  
  
1. Reduce cube map resolution to 1024 or 2048  
2. Use shared cube camera instead of per-object  
3. Update cube camera every 2-3 frames  
4. Check geometry complexity  
5. Only update when necessary  
  
For WebGL shaders:  
  
1. Reduce fragment shader complexity  
2. Minimize texture lookups  
3. Use lower precision where possible (mediump vs highp)  
4. Reduce number of loop iterations
```

Issue: Elements disappear after window resize

Ask Claude:

Claude, my game elements (aliens, obstacles, etc.) disappear when I resize the browser window.

Please implement proper `onResize()` handler:

1. Recreate all size-dependent elements (aliens, barriers, stars)
2. Reposition player/ship to center of new canvas
3. PRESERVE game state (score, lives, wave) during resize
4. Update boundaries for collision detection

Example pattern:

```
```javascript
onResize(width, height) {
 // Update positioning
 this.player.x = width / 2;
 this.player.y = height / 2;

 // Recreate elements (use helper methods that preserve state)
 this.recreateEnemies();
 this.recreateObstacles();
 this.recreateBackground();

 // Update boundaries
 this.leftBoundary = 50;
 this.rightBoundary = width - 50;
}

recreateEnemies() {
 // Save state
 const currentScore = this.score;
 const currentWave = this.wave;

 // Recreate
 this.createEnemies();

 // Restore state
```

```
this.score = currentScore;
this.wave = currentWave;
}
```

```
Issue: Game resets when resizing window
```

```
Ask Claude:
```

Claude, my game resets (score, lives, wave) when I resize the browser window.

The issue is that `onResize()` is calling create methods that reset game state.

Please create helper methods that preserve state during recreation:

```
recreateEnemies() {
 // Preserve state
 const savedScore = this.score;
 const savedLives = this.lives;
 const savedWave = this.wave;

 // Recreate enemies
 this.createEnemies();

 // Restore state
 this.score = savedScore;
 this.lives = savedLives;
 this.wave = savedWave;
}

onResize(width, height) {
 // Use state-preserving methods
 this.recreateEnemies();
 this.recreateBarriers();
}
```

```
Issue: Plugin flickers during window resize
```

```
Ask Claude:
```

Claude, my plugin flickers when dragging the window to resize.

The flickering is caused by the plugin resizing on every pixel change.

This is actually handled by Freque's global resize throttling in main.js, which delays plugin resizing by 150ms after the user stops dragging.

No changes to the plugin code are needed - the throttling is automatic!

However, please verify: 1. Canvas CSS uses width: 100% and height: 100% (set by base class) 2. onResize() doesn't set fixed pixel dimensions 3. Canvas is using percentage-based scaling

```

```

```
Advanced Prompting Techniques
```

```
Iterative Development
```

```
Start Simple
```

Claude, create a basic chrome sphere plugin with: - One sphere at center - Video reflections - Size slider - Audio-reactive scale

```
Then Add Features
```

Claude, update the sphere plugin to: - Add 5-20 spheres - Random positions - Floating animation - Beat-reactive pulse

### Then Polish

Claude, enhance the plugin with: - Dual mode (direct texture + reflection) - Background sphere visibility toggle - Dynamic cube camera positioning - Performance optimizations

## Combining References

Claude, create a plugin combining: - The cube camera system from Chrome Spheres - The particle emission from [another plugin] - The audio analysis from [another plugin] - Custom visual style: [describe]

## Requesting Specific Technologies

Claude, create a plugin using: - Three.js InstancedMesh for performance - Custom GLSL shaders for effects - Post-processing with bloom - Advanced PBR materials

---

## # Visual Description Library

### ## Movement Verbs

- **Float** - Gentle up/down motion
- **Drift** - Slow sideways motion
- **Pulse** - Rhythmic expand/contract
- **Spin** - Rotation on axis
- **Orbit** - Circle around point
- **Wave** - Undulating motion
- **Burst** - Sudden outward explosion
- **Spiral** - Circular with forward/back
- **Wobble** - Irregular shaking
- **Glide** - Smooth continuous motion

### ## Effect Words

- **Glow** - Soft light emanation
- **Trail** - Motion blur behind objects
- **Fade** - Gradual transparency
- **Shimmer** - Subtle sparkle
- **Ripple** - Expanding circular waves
- **Bloom** - Bright light overflow
- **Distortion** - Warped/twisted appearance
- **Reflection** - Mirror-like surface
- **Refraction** - Light bending
- **Dispersion** - Rainbow prismatic effect

### ## Color Descriptors

- **Gradient** - Smooth color transition
- **Vibrant** - Highly saturated

- **Pastel** - Soft, light colors
- **Monochrome** - Single color variations
- **Warm** - Reds, oranges, yellows
- **Cool** - Blues, greens, purples
- **Neon** - Bright, glowing colors
- **Metallic** - Chrome, gold, silver
- **Iridescent** - Rainbow-shifting colors

## ## Layout Terms

- **Center-focused** - Emanating from middle
- **Edge-emphasized** - Action at borders
- **Grid** - Organized rows/columns
- **Random** - Unpredictable placement
- **Circular** - Ring arrangement
- **Radial** - Spoke pattern from center
- **Layered** - Multiple depth levels
- **Scattered** - Loosely distributed
- **Clustered** - Grouped together

---

## # Real-World Examples

### ## Example 1: Chrome Spheres

**User Request:**

Claude, create floating chrome spheres that reflect the video/background.

Spheres: - 15-20 spheres - Different sizes - Floating animation - Chrome appearance

Video Reflections: - Use cube camera system - Support video and background image  
- Toggle between texture modes - Show/hide background sphere

Audio: - Bass → size - Mids → movement - Treble → shininess - Beat pulse

Controls: - Sphere count: 1-50 - Size range: 5-10 - Float speed: 0-10 - Metalness: 0-1  
- Roughness: 0-1 - Reflection intensity: 0-3

```
Result: Complete Chrome Spheres plugin with sophisticated cube camera system, dual mo
```

```
Example 2: Energy Particles
```

```
User Request:
```

Claude, create an energy particle system:

Particles burst from center on beats, drift outward, and fade. Colors shift with treble frequencies. Bass determines burst intensity. Mids control drift speed.

Controls: - Particle count - Burst size - Lifetime - Color scheme - Trail length

2 Presets: - Gentle: Small bursts, slow fade - Explosive: Large bursts, fast fade

```
Result: Beautiful particle system with beat detection, color shifting, and smooth tra
```

```
Example 3: Tunnel Vision
```

```
User Request:
```

Claude, create a 3D tunnel effect:

Camera moves through infinite tunnel of rings. Rings pulse with audio. Texture or video on rings. Bass makes tunnel expand. Beat creates shockwave effect.

Modes: - Geometric rings - Textured panels - Video mapped

Controls: - Speed - Ring spacing - Texture/video source - Pulse intensity

```
Result: Immersive 3D tunnel with multiple visual modes and strong audio reactivity.
```

```
Example 4: RETROE - Advanced Retro Tunnel Visualizer
```

```
User Request:
```

Claude, create an advanced retro arcade tunnel visualizer:

Plugin Name: RETROE Type: 2D Canvas

---

---

#### VISUAL DESCRIPTION

Main Elements: Neon wireframe rings that continuously spawn at center, grow larger as they move toward camera, and fade out at edges. Each ring lives for 3 seconds.

Movement Style: Smooth tunnel travel effect - rings spawn every 0.3 seconds and scale from small to large based on age. Rotation creates spiral motion.

Color Scheme: 10 retro palettes: Classic Arcade, Outrun Sunset, Matrix Green, Tron Blue, Cyberpunk, Arcade Cabinet, VHS Tape, CRT Monitor, Commodore 64, Game Boy

Special Effects: - VHS glitch with horizontal displacement and tracking errors - Enhanced RGB chromatic aberration (horizontal + vertical shift, edge fringing) - Film grain static with clustered noise and color artifacts - Scanlines for CRT effect - Screen curvature vignette - Pixelation (0-100) for low-res retro look - Color banding (0-100) for 8-bit posterization

---

---

#### AUDIO REACTIVITY

Master Control: - Beat React (Master ON/OFF toggle)

Core Reactivity (Toggles): - Audio → Color (ON by default): Colors morph with energy - Audio → Speed (OFF by default): Ring scale speed varies with energy - Audio → Segments (OFF by default): Segment count increases with energy

Global Controls: - Audio Sensitivity (0-200%): Global multiplier for all audio - Audio Smoothing (0-100%): Exponential smoothing to prevent jitter

Per-Feature Intensity: - Color Intensity (0-200%): How much audio affects color - Speed Intensity (0-200%): How much audio affects speed - Segment Intensity (0-200%): How much audio affects segments

Frequency-Specific (All OFF by default): - Bass → Rotation: Bass controls rotation speed - Bass → Spawn: Bass controls ring spawn rate - Mid → Glow: Mids control glow intensity - Treble → Thickness: Treble controls line thickness

---

---

## CONTROLS

Tunnel Controls: - Tunnel Speed (0-10): Base travel speed - Rotation Speed (0-10): Base rotation speed - Segment Count (8-48): Number of polygon sides - Ring Thickness (1-48): Line width - Glow Intensity (0-3): Neon glow amount - Scale (0.5-3): Overall visualization zoom - Background Opacity (0-1): Trail fade effect

Retro Effects: - VHS Glitch (0-1): Horizontal displacement intensity - RGB Shift (0-1): Chromatic aberration amount - Static/Noise (0-1): Film grain intensity - Screen Curve (0-1): CRT vignette strength - Pixelation (0-100): Low-resolution effect - Color Banding (0-100): Color depth reduction

Toggles: - Scanlines ON/OFF - Grid Lines ON/OFF - All audio reactivity toggles

---

---

## PRESETS

Preset 1 - Candy Squares: Purpose: Vibrant, colorful arcade feel Settings: High glow, thick rings, all effects maxed, arcade color scheme

Preset 2 - Classic Tunnel: Purpose: Balanced retro tunnel with smooth motion Settings: Medium speed, balanced effects, classic arcade colors

Preset 3 - VHS Nightmare: Purpose: Heavy glitch effects for intense visuals Settings: Max VHS glitch, high RGB shift, lots of static, outrun colors

Preset 4 - Game Boy Dream: Purpose: Authentic Game Boy green monochrome look Settings: Game Boy palette, high pixelation, color banding, low glow

---

---

## SPECIAL REQUESTS

Implementation Details: - Ring spawning system (age-based lifecycle, 0-1) - Post-processing effects (pixelation LAST, color banding before vignette) - Exponential audio smoothing to prevent jitter

- Frequency-specific reactivity with separate toggles - Per-feature intensity controls for fine-tuning - Enhanced VHS effects (vertical shift, edge fringing, color noise)

Performance: - Target 60fps smooth animation - Efficient ring management (remove dead rings) - Lazy-create pixel buffer only when needed

```
Result: Complete RETROE plugin with:
```

- ☒ Ring spawning system for smooth tunnel effect
- ☒ 10 authentic retro color schemes
- ☒ Advanced post-processing (pixelation, color banding)
- ☒ Granular audio controls (sensitivity, smoothing, per-feature intensity)
- ☒ Frequency-specific reactivity (bass/mid/treble toggles)
- ☒ Enhanced VHS/CRT effects
- ☒ Smooth 60fps performance
- ☒ 4 diverse presets covering different aesthetics

This example demonstrates requesting advanced features all in one comprehensive plugin.

```

```

```
Example 5: Tunnel Vision
```

```
Rings pulse with audio.
```

```
Texture or video on rings.
```

```
Bass makes tunnel expand.
```

```
Beat creates shockwave effect.
```

```
Modes:
```

- Geometric rings
- Textured panels
- Video mapped

```
Controls:
```

- Speed
- Ring spacing
- Texture/video source
- Pulse intensity

**Result:** Immersive 3D tunnel with multiple visual modes and strong audio reactivity.

---

# Tips for Success

---

## Do's

- ✓ **Be specific about visuals** - "Floating chrome spheres" is better than "3D objects"
- ✓ **Explain audio mapping** - Tell Claude exactly what affects what
- ✓ **Request proper Three.js setup** - Mention container sizing, CSS, encoding
- ✓ **Ask for cube cameras when needed** - For video reflections on metallic surfaces
- ✓ **Specify control types and ranges** - Give exact min/max values
- ✓ **Provide at least 2 presets** - Help define the plugin's range
- ✓ **Mention performance needs** - If targeting mobile or lower-end hardware
- ✓ **Ask questions if unclear** - Claude can help refine your vision

## Don'ts

- ✗ **Don't be vague** - "Make it cool" doesn't give enough guidance
- ✗ **Don't skip audio mapping** - Audio reactivity is core to Freque
- ✗ **Don't forget presets** - They're required for the plugin system
- ✗ **Don't ignore performance** - Mention if you need optimization
- ✗ **Don't assume** - Clearly state if you want video reflections

# Getting the Best Results

## 1. Start with Clear Vision

Think about what you want to see and how it should move.

## 2. Use the Template

The universal template ensures you cover everything.

## 3. Reference Examples

Mention other plugins or effects you like.

## 4. Iterate

Start simple, then ask Claude to add features.

## 5. Test and Refine

Try it, note issues, ask Claude to fix them.

---

# Conclusion

---

Claude is your partner in plugin creation. With clear descriptions and specific requirements, you can create professional-quality audio visualizations without deep coding knowledge.

**Remember:** - Use the universal template - Be specific about visuals and audio mapping - Request proper Three.js setup for 3D plugins - Ask for cube cameras when you want video reflections - Start simple and iterate - Don't hesitate to ask questions

**Happy creating!** 🚀🎨🎵

*Guide Version 2.0 - Updated with video reflection prompting*

---

## Freque Plugin Development Documentation

© 2025 Freque - Professional Audio Visualization Platform